Les tests (niveau * interface * angle * type)

Cette page définit les quatre axes essentiels pour décrire correctement une validation logicielle : le **niveau d'intégration**, l'**interface observée**, l'**angle d'observation** et le **type de validation**. L'objectif n'est pas de décider ici quoi automatiser, ni de reproduire une pyramide de tests, mais de donner un langage commun pour décrire précisément **ce qui est validé**, **à quelle frontière**, **avec quelle visibilité** et **pour quelle propriété attendue**.

> **Note terminologique :** cette page explique le cadre détaillé des validations logicielles : niveau d'intégration, interface observée, angle d'observation et type de validation. Les définitions courtes et transversales des termes sont maintenues dans le Lexique / Terminologies. Cette page précise leur usage opérationnel et pédagogique, sans remplacer le lexique comme source de référence.

> Un même type de test peut exister à plusieurs niveaux d'intégration. Par exemple, un test de performance, de sécurité, de résilience ou de compatibilité peut être réalisé en **small**, **medium** ou **large**, et tout autant en **black-box**, **white-box** ou **gray-box**, selon le périmètre et l'angle d'observation visés.

Cette distinction s'aligne avec l'approche Opal-RT où les spécifications doivent être **testables**, **traçables** et utilisables par toutes les disciplines, depuis DÉFINIR jusqu'à IMPLÉMENTER, DIFFUSER et CONFIRMER. Les niveaux ci-dessous servent donc à structurer la validation des comportements décrits dans les spécifications, et non à imposer une recette unique.

- Principe fondamental : quatre axes distincts
- Niveaux d'intégration
    - Clarifier les mots de la famille « intégrer »
  - Vue d'ensemble
  - Small
    - Pourquoi « small » plutôt que « unitaire »
  - Medium
  - Large
  - Small, medium et large dans trois interfaces
  - Exemples de combinaisons
- Angles d'observation : white-box, gray-box et black-box
- Ce que ces niveaux ne sont pas
- Types de validation
  - Liste des principaux types de test
  - Précisions utiles sur certains types
    - Tests de santé
    - Tests d'intégrité
    - Tests de limite
    - Tests de résilience

    - Tests UI, UX et utilisabilité
- Matrice de combinaison : niveau × type
- Exemples appliqués
- Lien avec les spécifications atomiques
- Repères pratiques d'utilisation
    - Quand choisir small
    - Quand choisir medium
    - Quand choisir large
- Erreurs fréquentes à éviter
- Résumé opérationnel
    - 📚 Références

Principe fondamental : quatre axes distincts

Un test ne devrait pas être décrit avec un seul mot comme « unitaire », « intégration », « E2E » ou « performance ». Pour être précis, il faut distinguer quatre axes :

- **Niveau d'intégration** : jusqu'où on assemble réellement pour valider un comportement.
- **Interface observée** : par quelle frontière on observe le système : interface utilisateur, interface applicative ou interface machine.
- **Angle d'observation** : avec quelle visibilité sur l'intérieur du système on réalise la validation : white-box, gray-box ou black-box.
- **Type de validation** : quelle propriété, qualité ou risque on cherche à observer : fonctionnel, performance, sécurité, résilience, intégrité, compatibilité, etc.

Autrement dit, on peut prendre un même comportement spécifié et le décrire comme une combinaison, par exemple : **medium × interface applicative × gray-box × contrat**, ou **large × interface utilisateur × black-box × fonctionnel**.

- **small**, **medium** ou **large** selon le périmètre réellement assemblé ;
- **interface utilisateur**, **interface applicative** ou **interface machine** selon la frontière observée ;
- **white-box**, **gray-box** ou **black-box** selon la visibilité disponible ;
- avec un type de validation comme **fonctionnel**, **performance**, **sécurité**, **résilience**, **contrat**, **intégrité** ou **compatibilité**.

> Il faut éviter de traiter **white-box**, **gray-box** et **black-box** comme des niveaux d'intégration. Ce sont des **approches d'observation**, pas des paliers d'assemblage du système.

Niveaux d’intégration

Les niveaux d’intégration représentent l’ampleur du système impliqué pour valider une spécification. Ils ne décrivent pas seulement des tests techniques ou du code : ils décrivent **jusqu’où on assemble** pour observer un comportement attendu, que ce soit dans l’interface utilisateur, l’interface applicative ou l’interface machine.

Clarifier les mots de la famille « intégrer »

Avant de parler de niveaux d’intégration, il est utile de distinguer quelques mots proches qui sont souvent mélangés, mais qui ne désignent pas la même chose.

Les définitions de référence de **intégration**, **intégré**, **intégralité**, **intégrité**, **intégral** et **intègre** sont maintenues dans le Lexique / Terminologies. Dans cette page, le mot **intégration** est utilisé principalement au sens opérationnel suivant : assembler des éléments distincts et valider que leur collaboration produit le comportement attendu.

Cette distinction permet ensuite de comprendre les niveaux **small**, **medium** et **large** sans confondre intégration, intégrité ou intégralité d’un périmètre.

Pour vulgariser au maximum la différence entre les niveaux, on peut représenter une interaction simple entre deux éléments **A** et **B** :

- **Small** : on valide le composant lui-même, donc **A** ou **B** isolément.
- **Medium** : on valide l’aller entre deux éléments, donc **A → B**.
- **Large** : on valide l’aller-retour complet et le résultat observable, donc **A → B → A**, souvent dans une logique E2E.

Cette représentation est volontairement simplifiée. Elle sert à mieux visualiser le périmètre d’intégration : small isole une partie, medium valide une interaction, et large confirme qu’un flux complet revient avec le résultat attendu.

Vue d’ensemble

Niveau	Définition	Périmètre typique	But principal
Small	Validation du plus petit comportement isolable.	Fonction, règle, composant UI, service, module logique ou élément machine isolé.	Confirmer qu’un comportement atomique est correct sans dépendre d’un assemblage plus large.

Medium	Validation de l'intégration entre plusieurs éléments small.	Deux ou plusieurs composants, services, modules ou couches en interaction.	Confirmer que les contrats, échanges et dépendances fonctionnent ensemble.
Large	Validation d'un ensemble complet orienté usage ou mission.	Chaîne complète, scénario utilisateur, système bout en bout, environnement quasi réel ou réel.	Confirmer qu'un flux complet produit le résultat attendu du point de vue du système ou de l'utilisateur.

Small

Small correspond au plus petit comportement qu'on peut valider de façon claire et isolée. Ce n'est pas réellement un niveau d'intégration au sens strict, car on n'est pas encore dans l'assemblage significatif de plusieurs éléments; c'est plutôt le **plus petit divisible testable**, peu importe qu'il soit situé dans le code, l'interface utilisateur, l'interface applicative ou l'interface machine.

Pourquoi « small » plutôt que « unitaire »

Traditionnellement, ce niveau est souvent associé au **test unitaire**. Le terme reste utile dans plusieurs contextes, mais il est devenu ambigu : selon les équipes, il peut désigner un niveau d'intégration, un style de test, un outil, ou simplement une habitude de développement.

Par exemple, on peut faire des tests unitaires de code sur une fonction ou une classe, mais aussi des tests unitaires UI sur un composant visuel isolé. À l'inverse, un test écrit avec un outil comme JUnit peut très bien exercer un scénario beaucoup plus large, incluant une base de données, des services ou même un flux quasi E2E. Le nom de l'outil ne suffit donc pas à déterminer le niveau réel du test.

C'est pourquoi la nomenclature **small / medium / large** est plus précise : elle décrit le **périmètre d'intégration réellement observé**, indépendamment de la technologie, du framework ou du nom historique donné au test.

On l'utilise pour valider :

- une règle métier atomique ;
- un calcul ;
- une transformation de données ;

- une validation d'entrée ;
- une décision locale ;
- un comportement attendu d'une unité logique ;
- un composant UI isolé ;
- un script, un agent, un driver ou un module machine isolé.

Le niveau small permet de répondre à la question : **est-ce que cette partie isolée fait exactement ce qu'elle doit faire ?**

Medium

Medium correspond à une validation d'intégration entre plusieurs éléments small. On ne valide plus seulement un comportement isolé, mais la **collaboration** entre au moins deux parties significatives.

On l'utilise notamment pour valider :

- un appel entre deux modules ;
- un contrat d'API ;
- une interaction entre couches applicatives ;
- un échange de données entre composants ;
- un traitement qui dépend d'une base de données, d'un service ou d'un adaptateur ;
- une orchestration partielle.

Le niveau medium permet de répondre à la question : **est-ce que ces éléments s'intègrent correctement entre eux ?**

Large

Large correspond à la validation d'un ensemble cohérent de comportements assemblés dans un but précis. Ici, on cherche à observer un **flux complet** qui envoie quelque chose au système et obtient une réponse ou un résultat observable.

On parle souvent ici de **bout en bout** (*end-to-end*, E2E), mais le terme large est plus large que le seul parcours interface utilisateur : il peut couvrir tout scénario complet orienté mission, système ou client.

On l'utilise pour valider :

- un parcours utilisateur complet ;
- un scénario inter-systèmes ;
- une chaîne de traitement de bout en bout ;

- une opération réelle ou quasi réelle avec ses dépendances importantes ;
- une confirmation qu'un objectif métier ou système est atteint.

Le niveau large permet de répondre à la question : **est-ce que l'ensemble du comportement attendu fonctionne vraiment dans son contexte global ?**

Small, medium et large dans trois interfaces

Pour éviter l'ambiguïté du terme « unitaire », les niveaux **small**, **medium** et **large** peuvent être appliqués à plusieurs interfaces du système. On ne parle donc pas seulement de code : on peut raisonner sur l'interface utilisateur, l'interface applicative et l'interface machine.

Interface	Small	Medium	Large
Interface utilisateur	Composant visuel isolé, règle d'affichage locale ou interaction élémentaire.	Interaction entre vues, navigation partielle, état partagé entre composants.	Parcours utilisateur complet, scénario bout en bout observable dans l'interface.
Interface applicative	Fonction, service, règle métier, transformation ou contrat local isolé.	Contrat entre services, échange entre couches, orchestration partielle.	Chaîne complète d'appels, flux transactionnel ou scénario applicatif de bout en bout.
Interface machine	Script, agent, driver, module système ou comportement machine isolé.	Intégration entre composants système, protocole, communication inter-processus ou dépendance matérielle simulée.	Système complet déployé, environnement quasi réel, simulation temps réel ou scénario machine complet.

Cette grille donne **3 niveaux × 3 interfaces**. Elle permet de situer précisément une validation, de modéliser les scénarios d'intégration, d'identifier les contrats à respecter, de prévoir les mappings nécessaires et de choisir les mocks ou simulateurs appropriés.

Exemples de combinaisons



Les exemples détaillés de combinaisons **niveau × interface × angle × type** sont maintenus dans la page dédiée Exemples concrets de combinaisons. Cette page-ci garde le cadre conceptuel; la page d'exemples sert de compagnon pratique et peut évoluer avec de nouveaux cas concrets.

Angles d'observation : white-box, gray-box et black-box

Les notions **white-box**, **gray-box** et **black-box** décrivent le degré de visibilité sur l'intérieur du système pendant la validation. Elles peuvent s'appliquer à **n'importe quel niveau**.

Approche	Ce qu'on connaît	Usage typique
White-box	La structure interne, le code, les branches, les états ou les mécanismes internes sont connus et exploités.	Validation fine d'algorithmes, chemins logiques, conditions internes, instrumentation technique.
Gray-box	Une partie de l'interne est connue, sans tout exposer ni tout piloter.	Validation d'intégration, observation d'états intermédiaires, diagnostic de comportements, contrats techniques.
Black-box	On observe uniquement les entrées, sorties et effets visibles, sans se baser sur l'implémentation interne.	Validation fonctionnelle, comportement utilisateur, conformité aux spécifications observables.

Une même spécification peut être validée en **black-box large** pour un parcours complet utilisateur, puis en **white-box small** pour sécuriser une règle critique, puis en **gray-box medium** pour diagnostiquer l'intégration entre deux services.

Ce que ces niveaux ne sont pas

- Ils ne sont pas une stratégie d'automatisation à eux seuls.
- Ils ne remplacent pas le jugement qualité ni l'analyse de risque.
- Ils ne disent pas automatiquement quels tests sont manuels ou automatisés.

- Ils ne sont pas synonymes de white-box, gray-box ou black-box.
- Ils ne doivent pas être réduits à une simple pyramide de tests.

La pyramide peut parfois servir à discuter d'un **équilibre d'automatisation**, mais elle ne définit pas les niveaux d'intégration. Ici, on cherche avant tout à nommer correctement **le périmètre validé**.

⚠ Confondre **niveau d'intégration**, **type de test** et **niveau d'automatisation** mène à des discussions floues, à des trous de couverture et à une traçabilité faible entre spécifications et validation.

Types de validation

Les types de validation décrivent **ce qu'on cherche à apprendre ou à prouver** à partir d'une spécification. Cette page en présente les principaux pour expliquer le modèle global, mais le catalogue détaillé des catégories et types de tests est maintenu dans une page dédiée.

Liste des principaux types de test

Type	Description	Question à laquelle il répond
Fonctionnel	Vérifie que le comportement livré correspond aux **spécifications IF... THEN...** et aux règles applicables.	Est-ce que le système fait ce qui est spécifié ?
Santé	Validation rapide et ciblée d'une capacité essentielle pour confirmer qu'un système est utilisable à un niveau minimal.	Le système semble-t-il vivant et globalement utilisable ?
Intégrité	Vérifie la cohérence, la complétude et la non-corruption des données, états, traitements ou artefacts.	Les données et états restent-ils cohérents et fiables ?
Régression	Vérifie qu'un changement n'a pas dégradé des comportements déjà conformes.	Avons-nous cassé quelque chose qui fonctionnait déjà ?
Limite	Vérifie le comportement aux frontières : tailles maximales, minimums, seuils, valeurs extrêmes, saturation.	Le système tient-il correctement aux bornes de son domaine ?

Résilience	Vérifie la capacité à continuer, se dégrader proprement, récupérer ou tolérer des perturbations.	Que se passe-t-il quand tout ne va pas comme prévu ?
Performance	Mesure le temps de réponse, le débit, la latence, l'utilisation des ressources ou la tenue sous charge.	Le système est-il assez rapide et stable pour l'usage visé ?
Sécurité	Vérifie la protection contre les accès non autorisés, les vulnérabilités, les fuites, les élévations de privilèges et usages malveillants.	Le système protège-t-il correctement ses actifs et ses utilisateurs ?
Compatibilité	Vérifie le comportement sur différents environnements, systèmes, navigateurs, versions, matériels ou dépendances.	Le comportement reste-t-il correct dans les contextes supportés ?
Installation / déploiement	Vérifie l'installation, la mise à jour, la configuration, le rollback et le démarrage opérationnel.	Peut-on livrer, installer et remettre en état correctement ?
UI	Vérifie l'interface utilisateur : rendu, interactions, états visuels, composants, navigation visible.	L'interface se comporte-t-elle comme attendu ?
UX	Vérifie l'expérience d'usage : compréhension, fluidité, friction, enchaînement, clarté, efficacité perçue.	L'expérience utilisateur fait-elle du sens et soutient-elle l'objectif ?
Utilisabilité	Évalue la facilité d'apprentissage, d'exécution, de compréhension et de prévention des erreurs pour le profil visé.	Un utilisateur cible peut-il utiliser le produit correctement et sans effort excessif ?

Précisions utiles sur certains types

Tests de santé

Les tests de santé servent à confirmer rapidement qu'une capacité essentielle répond encore. Ils sont utiles après un déploiement, une intégration importante ou une correction urgente. Ils ne remplacent pas une validation complète.

Tests d’intégrité

Les tests d’intégrité visent la cohérence du système : données non corrompues, transitions valides, référentiels alignés, résultats non contradictoires, absence de perte silencieuse.

Tests de limite

Les limites ne concernent pas seulement des chiffres extrêmes. Elles incluent aussi les tailles de lots, les durées, les nombres d’utilisateurs, les capacités mémoire, les frontières de formats, les transitions d’état inhabituelles et les seuils métier.

Tests de résilience

La résilience vérifie le comportement en présence d’échecs : réseau indisponible, dépendance lente, service absent, redémarrage, données partielles, compétition d’événements, reprise après incident.

Tests UI, UX et utilisabilité

Ces trois dimensions sont liées mais différentes :

- **UI** : l’interface se rend-elle et réagit-elle correctement ?
- **UX** : l’enchaînement global est-il logique, fluide et pertinent ?
- **Utilisabilité** : un utilisateur cible peut-il accomplir sa tâche avec clarté et efficacité ?

Matrice de combinaison : niveau × type

Les types de test peuvent être appliqués à tous les niveaux. La bonne question n’est donc pas « *ce type appartient-il à ce niveau ?* », mais plutôt « *à quel niveau ce type apporte-t-il la preuve la plus utile ?* ».

Type de validation	En small	En medium	En large
Fonctionnel	Règle isolée correcte.	Contrat entre composants correct.	Flux complet conforme aux attentes.
Performance	Coût d’un algorithme ou d’une fonction.	Temps d’échange entre modules ou services.	Temps de réponse global et tenue sous charge.
Sécurité	Validation locale d’autorisations ou contrôles.	Contrats sécurisés entre couches ou services.	Scénario complet d’accès, session, permissions et exposition.

Résilience	Gestion locale d'erreur ou de repli.	Comportement entre dépendances en cas de panne.	Comportement global du système en situation dégradée.
Compatibilité	Composant compatible avec un runtime ou format précis.	Interaction compatible entre versions ou interfaces.	Parcours complet compatible avec l'environnement cible.
UI / UX / utilisabilité	Composant ou comportement visuel local.	Interaction entre vues, états et navigation partielle.	Expérience complète utilisateur du début à la fin.

Exemples appliqués

> Les exemples appliqués, dont l'exemple d'application web, sont maintenus dans la page dédiée Exemples concrets de combinaisons. Cette page-ci conserve le cadre conceptuel afin d'éviter de dupliquer les exemples et de maintenir une seule source pratique à jour.

Lien avec les spécifications atomiques

Dans l'approche DÉFINIR, chaque spécification atomique doit permettre de dériver clairement :

- le **small** minimal à valider ;
- les **intégrations medium** à sécuriser ;
- les **scénarios large** qui prouvent la valeur complète ;
- les **types de validation** pertinents selon les risques fonctionnels et non fonctionnels.

Une bonne spécification ne dit pas seulement *quoi construire* ; elle rend visible *comment prouver que c'est correct*.

> Une spécification de qualité permet de relier explicitement **besoin → spécification → niveaux de validation → types de test → résultats**.

Repères pratiques d'utilisation

Quand choisir small

- quand le comportement est atomique et isolable ;
- quand on veut un diagnostic rapide et précis ;

- quand on veut sécuriser une règle critique au plus près ;
- quand on veut réduire le bruit causé par des dépendances externes.

Quand choisir medium

- quand le risque principal est dans l'intégration ;
- quand plusieurs composants doivent coopérer ;
- quand le contrat entre modules, services ou couches est critique ;
- quand un défaut n'apparaît qu'au contact de dépendances réelles ou simulées.

Quand choisir large

- quand il faut prouver la valeur complète ;
- quand l'observable important est côté utilisateur ou système global ;
- quand la décision de livraison exige une preuve bout en bout ;
- quand les risques majeurs émergent seulement dans le flux complet.

Erreurs fréquentes à éviter

- Utiliser le mot **unitaire** sans préciser le périmètre réel : small UI, small applicatif, small machine, ou au contraire validation déjà medium ou large.
- Supposer que **large** signifie toujours interface graphique seulement.
- Croire qu'un test **black-box** est forcément **large**.
- Confondre **type de test** et **niveau d'intégration**.
- Réduire la discussion à l'automatisation au lieu de parler d'abord du besoin de preuve.
- Oublier que les aspects non fonctionnels doivent aussi être dérivés des spécifications.

Résumé opérationnel

Question	Réponse courte
Que sont les niveaux d'intégration ?	Le périmètre assemblé du système qu'on valide : small, medium, large.
Que sont les types de test ?	L'angle de qualité ou de risque qu'on cherche à valider : fonctionnel, sécurité, performance, résilience, etc.
Que sont white-box, gray-box, black-box ?	Des approches d'observation applicables à n'importe quel niveau.

Peut-on faire de la performance ou de la sécurité en small ?	Oui. Ces types de validation peuvent exister à tous les niveaux.
La pyramide de tests définit-elle ces niveaux ?	Non. Elle peut aider à parler d'automatisation, mais elle ne définit pas les niveaux d'intégration.

📚 Références

- Lexique / Terminologies — définitions de référence des termes utilisés dans ce cadre.
- Exemples concrets de combinaisons (niveau × interface × angle × type) — exemples pratiques appliquant le cadre de cette page.
- SDLC officiel OPAL-RT — cadre global dans lequel les validations s'inscrivent, de DÉFINIR à CONFIRMER.
- Contrat qualité officiel OPAL-RT — exigences et règles qualité auxquelles les validations doivent contribuer.
- RACI officiel OPAL-RT — rôles et responsabilités associés à la définition, l'exécution, la revue et l'approbation des validations.
- Qualimétrie officielle OPAL-RT — vision officielle de la mesure de la qualité et de l'amélioration continue.
- Les métriques de la qualimétrie — référentiel opérationnel des KPI et signaux liés à la qualité des validations.
- SPÉCIFICATION — cadre de référence pour formuler des spécifications IF… THEN… testables et traçables.
- Comment définir — pratiques de définition qui rendent les validations dérivables des spécifications.
- Formation continue — parcours d'adoption des pratiques SDLC, de spécification, de validation et de qualimétrie.